

Project Description:

This project involved designing and implementing an SQL database to transform raw genomic and clinical data into a system that can be queried for biological insight in patients with lung squamous cell carcinoma (LUSC). Raw patient, sample, mutation, and expression datasets were cleaned and standardized using Python scripts before being populated into the database. The final database organizes information across all datasets. Linux command line tools and SQL scripts were used to automate database creation and population. The completed database allows for querying of biologically relevant relationships like identifying mutated genes, examining mutation impact, and connecting mutations across different patient samples.

Data Sources:

The data came from CBioPortal: <https://www.cbioportal.org/>. All of this data specifically came from the TCGA Nature 2012 files of Lung Squamous Cell Carcinoma. All files used were originally in the .txt format.

Data Cleaning:

Files:

- data_clinical_patient.txt:
 - Deletes all metadata rows
 - All columns were kept
 - All column names were standardized into lowercase letters
 - Replaced any empty values with NaN
 - Converted data types in age and pack_years to float
 - Standardized sex values (Male → M)
 - Normalized smoking status categories:
 - Current smoker → current
 - Current reformed smoker for ≤15 years → former_recent
 - Current reformed smoker for >15 years → former_long
 - Lifelong Non-smoker → never
 - Missing → unknown
 - Handling pack_years values:
 - If smoking_status = never = 0
 - Missing pack_years = median pack_years\
 - Missing age values get filled with median age
 - Replaced "MX" in m_stage with missing → missing staging values get replaced with "Unknown"
 - Simplifying Stage Categories
 - T2a/T2b → T2
- data_clinical_sample.txt:
 - Delete all metadata rows
 - Columns kept:

- PATIENT_ID
 - SAMPLE_ID
 - NONSENSE_MUTATION
 - NONSILENT_PER_MB
 - TMB_NONSYNONYMOUS
- Standardized column names
- data_mutations.txt
 - Removed metadata
 - Standardized column names
 - Columns kept
 - Hugo_Symbol
 - Tumor_Sample_Barcode
 - Chromosome
 - Start_Position
 - End_Position
 - Variant_Classification
 - Variant_Type
 - IMPACT
 - Reference_Allele
 - Tumor_Seq_Allele2
 - HGVS
 - SIFT
 - PolyPhen
- All other columns removed because they were not needed for the scope of this database. With these columns you can see which genes are mutated and highly expressed. You can also see sample relationships like those with similar mutation profiles or which mutations have high impact across many patients.

Database Design:

- The design separates stable biological entities from observations. Patients, samples, genes, variants, annotations, expression values, and mutation calls are separated so data is not duplicated unnecessarily. For example, a variant is defined once in Variant, then connected to many samples through Sample_Mutation.
- Patient details are not repeated for every mutation, gene names are not repeated for every expression or variant record, and annotations are separated from variant identity.
- Assumptions:
 - A patient can have multiple samples but each sample only belongs to one patient
 - A gene can have many variants, variant belongs to one gene
 - A variant can exist in many samples
 - Each gene/sample pair only has one expression value
 - Each variant has zero or one annotation row
 - Each variant is uniquely identified by its gene, chromosome, start/end pos., ref allele, tumor allele, and variant type

- Business rules
 - Every patient must have a unique Patient_ID, as well as sample with unique Sample_ID and belong to an existing patient
 - Similarly Gene must have unique HUGO symbol and variant must belong to a gene

Normal Form/Deviations:

This database mostly satisfies 3NF. Patient, Samples, Genes, Variant, Variant_Annotation, Expression, and Sample_Mutation each represent a distinct entity or relationship to avoid duplication. Samples have tumor stages there because a patient can come in more than once and have multiple samples of different stages. Variant is separate from Sample_Mutation because a genomic variant can appear in many samples. Expression uses a composite key because expression is naturally a measurement for one gene in one sample.

Relationships/Constraints:

Primary Keys

- Patients → PATIENT_ID
- Samples → SAMPLE_ID
- Genes → GENE_ID
- Variant → VARIANT_ID
- Variant_Annotation → VARIANT_ID
- Expression → GENE_ID, SAMPLE_ID (composite)
 - One expression value per sample/gene pair
- Sample_Mutation → SAMPLE_MUTATION_ID

Foreign Keys

- Sample → PATIENT_ID
 - Ensures every sample belongs to a patient
- Variant → GENE_ID
 - Ensures every variant belongs to a gene
- Variant_Annotation → VARIANT_ID
 - An variant annotation cannot exist without a variant
- Expression → GENE_ID, SAMPLE_ID
 - Guarantees that expression measurements only exist for valid genes and samples
- Sample_Mutation → SAMPLE_ID, VARIANT_ID
 - A mutation can only exist if sample and variant are there

Unique Constraints

- Genes → UNIQUE(hugo_symbol)
 - Ensures a hugo symbol can only appear once

- Variant → UNIQUE(GENE_ID, Chromosome, Start_Position, End_Position, Reference_Allele, Tumor_Seq_Allele2, Variant_Type)
 - Prevents storing the same mutation multiple times
 - Variants are reusable for other samples (no duplication)

Nulls

- All primary/foreign keys → they define identity and relationships so they shouldn't be NULL

Data Dictionary:

Patients Table

- PATIENT_ID [VARCHAR(20)] → Primary Key, Unique patient identifier
- SEX [VARCHAR(10)] → Patient sex
- SMOKING_HISTORY [VARCHAR(70)] → smoking status/history
- SMOKING_PACK_YEARS [DECIMAL(5,2)] → estimated smoking pack in years

Samples Table

- SAMPLE_ID [VARCHAR(25)] → Primary Key, Unique sample identifier
- PATIENT_ID [VARCHAR(20)] → Foreign Key, patient linked to sample
- AGE [TINYINT] → patient age at time of sample collection
- NONSENSE_MUTATION [INT] → count of nonsense mutations
- NONSILENT_PER_MB [DECIMAL(5,2)] → nonsilent mutation rate per megabase
- TMB_NONSYNONYMOUS [DECIMAL(10,4)] → tumor mutation burden for nonsynonymous mutations
- CLIN_T_STAGE [VARCHAR(10)] → clinical tumor stage
- CLIN_N_STAGE [VARCHAR(10)] → clinical lymph node stage
- CLIN_M_STAGE [VARCHAR(10)] → clinical metastasis stage

Genes Table

- GENE_ID [INT] → Auto increment, primary key, unique gene identifier
- hugo_symbol [VARCHAR(20)] → unique, official gene symbol

Variant Table

- VARIANT_ID [INT] → primary key, unique variant identifier
- GENE_ID [INT] → foreign key, unique composite key, gene associated with variant
- Chromosome [VARCHAR(5)] → unique composite key, chromosome where variant occurs
- Start_Position [INT] → unique composite key, genomic start coordinate
- End_Position [INT] → unique composite key, genomic end coordinate
- Reference_Allele [VARCHAR(50)] → unique composite key, reference allele
- Tumor_Seq_Allele2 [VARCHAR(50)] → unique composite key, tumor alternate allele
- Variant_Type [VARCHAR(20)] → unique composite key, type of variant (SNP, DEL, etc...)

Variant_Annotation Table

- VARIANT_ID [INT] → primary/foreign key, variant being annotated
- HGCSp_Short [VARCHAR(50)] → short protein level variant annotation
- SIFT [VARCHAR(50)] → SIFT functional prediction
- PolyPhen [VARCHAR(50)] → PolyPhen functional prediction

Expression Table

- GENE_ID [INT] → composite primary key/foreign key, gene being measured
- SAMPLE_ID [VARCHAR(25)] → composite primary key/foreign key, sample being measured
- RPKM_VALUE [DECIMAL(12,6)] → gene expression value in RPKM

Sample_Mutation

- SAMPLE_MUTATION_ID [INT] → Auto increment, primary key, unique identifier for a sample mutation
- SAMPLE_ID [VARCHAR(25)] → foreign key, sample containing mutation
- VARIANT_ID [INT] → foreign key, variant observed in sample
- Variant_Classification [VARCHAR(30)] → function class of mutation
- IMPACT [VARCHAR(20)] → predicted mutation impact level

Script Map:

Python Script Run Order:

- 1) 1_cleaningpatientfile.py
 - It first loads the file and skips the metadata rows that start with #
 - Standardizes the column names into lowercase
 - Replaces blank strings with missing values
 - Converts numeric columns like age and pack years
 - Standardizes sex values like male and female or unknown
 - Normalizes smoking status
 - Fills pack years for never smokers with 0
 - Fills missing numeric values with median values
 - Cleans/simplifies staging columns
 - Lastly saves the cleaned dataset as CSV
- 2) 2_cleaningsamplefile.py
 - Reads the data in, ignoring metadata rows that start with #
 - Converts all columns to lowercase
 - Only keeps desired columns
 - Ensures numeric columns are floats
 - Saves CSV
- 3) 3_cleaningmutationfile.py
 - Reads the data, ignoring metadata rows #
 - Keeps only desired columns
 - Converts all columns names to lowercase

- Saves CSV
- 4) 4_patient_sample_inserts.py
 - This script creates the SQL inserts from the cleaned patient and sample files
- 5) 5_mutationsinserts.py
 - This script creates the SQL inserts from the cleaned mutation file
 - It checks to see if all required columns are there
 - Converts python values into sql safe strings with def sql_str
 - Def sql_num converts numbers safely and handles NULLs, floats vs. integers
- 6) 6_expressioninserts.py
 - Inserts the expression data from the data_mrna_seq_rpkm.txt file
 - Also uses the cleaned mutation file to keep track of which genes are in that file
 - This script only makes inserts for genes that are in the mutation file
- 7) 7_comparinghugos.py
 - This extra script is to compare the genes seen in both the mutations file and the expression file
 - Prints output that lists genes only in mutations file, and then genes only in the expression file

SQL Order:

- 1) 1_creatingtables.sql
 - This creates the main tables in the database
- 2) 2_adjusted_tables.sql
 - This adds in the unique key for the Variant table so that there are no duplicates
- 3) 3_insert_pt_spl.sql
 - This populates the Patients and Samples tables
- 4) 4_mutations_inserts.sql
 - This populates the Genes, Sample_Mutation, Variant and Variant_Annotation tables
- 5) 5_expression_inserts_filtered.sql
 - This populates the Expression table
- 6) 6_filtered_no_expressions
 - This filters the tables to only include genes that are both in the data_mutations_cleaned.csv file and data_mrna_seq_rpkm.txt file
 - Before this there were genes in the database that were only in the mutations file that were not in the expressions file

Reproduction Instructions:

Software Needed:

- PyCharm (not needed but it is what was used here)
- Python
 - Packages: pandas and math
- MySQL

- VirtualBox BIN6970 Linux

Reproduction Instructions:

- 1) With the DBProject Folder there are 5 main folders: Data, Diagrams, Docs, Scripts and SQL
- 2) Clean the raw files in DBProject > Data > Raw with python scripts 1-3. Running it in PyCharm or IDLE.

This will produce the cleaned files:

- data_clinical_patient_cleaned.csv
 - data_mutations_cleaned.csv
 - Data_clinical_sample_cleaned.csv
- 3) Compare the mutation genes with expression genes with script 7
 - a) A report with the comparison will be generated in a txt file
 - 4) Generate the SQL insert files using the python scripts 4-6
These SQL files will be generated:
 - 3_insert_pt_spl.sql
 - 4_mutations_inserts.sql
 - 5_expression_inserts_filtered.sql
 - 5) On your local browser go to: <http://localhost:8888/>
 - a) This should get you on to phpMyAdmin
 - b) Create a new database by clicking New and name it DBProject
 - 6) In the linux command line on virtual box
 - a) Use:
 - i) `mysql -h 127.0.0.1 -P 3306 -u root -pinstructor`
 - ii) use DBProject
 - iii) Source /media/sf_DBProject/(file path for inserts)
Ex: Source /media/sf_DBProject/SQL/1_creatingtables.sql
(1) Do this step for each SQL file in numbered order
 - 7) Wait and verify that the tables have populated correctly
 - a) Check to see the correct number of patients have populated
 - b) Can also check the number of rows in the Genes table, should match numbers seen in the earlier file produced from python script 7

Limitations and future work:

- Database depends on externally provided clinical, mutation, and expression datasets
 - There are some inconsistencies, missing values, or annotation errors in the original files that have to be explicitly cleaned otherwise it'll show up in the database
 - The mutations and expression files did not contain a perfect amount of overlapping genes; some were in one and not the other, which were then removed at the end when running the SQL scripts.
 - This database is relatively simple and only includes data on patients, samples, mutations, gene expression and variant annotations. It does not include things like methylation data, survival outcomes, treatment history or proteomic data

- Variants are also only identified by chromosome, positions, alleles, type, and associated genes. It does not include multiple alternate alleles, structural variants, or complex rearrangements.
- Expression value is also stored at RPKM, which are not raw read count values
- Future additions
 - Methylation, proteomics, pathway enrichment and TF activity could be added for a more complete multi-omic systems biology analysis
 - Other additions like ClinVar, dbSNP, Ensembl, gnomAD could also be added for more biological interpretations and clinical relevance
 - A better user interface could be used as well for ease of use and better visualization